

# UAV Flight Analyzer

|                |                                       |
|----------------|---------------------------------------|
| Version        | 1.0                                   |
| Date           | April 2026                            |
| Platform       | ArduPilot DataFlash .BIN Log Analysis |
| Modules        | 18 Analysis Modules                   |
| Vehicle Types  | Quadcopter • VTOL • Fixed-Wing        |
| Output Formats | HTML Report • JSON • Fleet SQLite     |

## UAV Flight Analyzer — Technical Documentation & User Manual

**Version:** 1.0  
**Date:** April 2026  
**Platform:** ArduPilot DataFlash .BIN Log Analysis

---

# Table of Contents

---

1. [Introduction](#)
  2. [Installation & Setup](#)
  3. [Quick Start Workflow](#)
  4. [Web UI — Analyze Tab](#)
  5. [Web UI — Fleet Dashboard](#)
  6. [Vehicle Profiles & YAML Reference](#)
  7. [Analysis Modules Reference](#)
  8. [Scoring & Grading System](#)
  9. [Fleet & Trend Monitoring](#)
  10. [CLI Reference](#)
  11. [API Reference](#)
  12. [Troubleshooting & FAQ](#)
  13. [Glossary](#)
-

# 1. Introduction

## 1.1 What Is UAV Flight Analyzer?

UAV Flight Analyzer is a **local web application** for analyzing ArduPilot flight logs. It reads raw DataFlash `.BIN` files directly from the flight controller SD card, runs 18 independent analysis modules across every critical flight health domain, assigns a weighted score from 0 to 100, and generates professional HTML and JSON reports.

No data leaves your machine. No internet connection is required. All processing, storage, and reporting is local.

## 1.2 Problems It Solves

| Problem                               | How the Tool Addresses It                              |
|---------------------------------------|--------------------------------------------------------|
| Manual log review takes too long      | Automated scoring and issue detection in seconds       |
| Maintenance decisions are subjective  | Standardized issue codes and thresholds                |
| Hard to spot fleet-wide deterioration | Per-drone trend tracking with linear regression alerts |
| Difficult to compare airframes        | Weighted scoring across 18 domains in one number       |
| Reports not shareable                 | Self-contained HTML report, downloadable JSON          |

## 1.3 Target Users

- **Flight Operators** — review each mission log before the next flight
- **Maintenance Engineers** — act on critical and warning issue codes
- **Fleet Managers** — monitor trends across multiple drones
- **Integration Developers** — automate analysis via REST API

## 1.4 System Requirements

| Requirement | Minimum |
|-------------|---------|
|-------------|---------|

|                  |                                            |
|------------------|--------------------------------------------|
| Python           | 3.10 or newer                              |
| Operating System | Windows 10/11, macOS 12+, Ubuntu 20.04+    |
| RAM              | 512 MB free                                |
| Input Format     | ArduPilot DataFlash <code>.BIN</code> only |
| Browser          | Chrome, Firefox, Edge, Safari (for web UI) |

---

## 2. Installation & Setup

### 2.1 Clone the Repository

```
git clone https://github.com/NAGASURYA444/UAV-Flight-Analyzer.git
cd UAV-Flight-Analyzer
```

### 2.2 Create a Virtual Environment (Recommended)

```
python -m venv venv
```

Activate the environment:

**Windows:**

```
venv\Scripts\activate
```

**macOS / Linux:**

```
source venv/bin/activate
```

### 2.3 Install Dependencies

```
pip install -r requirements.txt
```

Key dependencies installed:

| Package       | Purpose                               |
|---------------|---------------------------------------|
| pymavlink     | ArduPilot DataFlash .BIN parser       |
| pandas        | Telemetry data processing             |
| numpy / scipy | Statistical analysis, FFT, regression |
| pyyaml        | Vehicle profile YAML loading          |

---

|                                |                                   |
|--------------------------------|-----------------------------------|
| <code>fastapi / uvicorn</code> | Web server and REST API           |
| <code>click / rich</code>      | CLI interface and terminal output |
| <code>python-multipart</code>  | File upload handling              |

---

## 2.4 Start the Web Server

```
python run_web.py
```

Open your browser at: **http://localhost:5000**

The server is ready when you see:

```
INFO:      Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
```

## 2.5 Daily Use

After first-time setup, just activate and run:

```
# Windows
venv\Scripts\activate
python run_web.py

# macOS / Linux
source venv/bin/activate
python run_web.py
```

---

## 3. Quick Start Workflow

---

### 3.1 Web UI Path (Recommended)

```
Step 1 → Open http://localhost:5000 in your browser
Step 2 → Click the Analyze tab
Step 3 → Enter a Drone ID (e.g. "UAV-001")
Step 4 → Select Vehicle Type (Quadcopter / VTOL / Fixed-Wing)
Step 5 → Drag and drop your .BIN file onto the upload zone
Step 6 → (Optional) Expand Profile Settings and adjust thresholds
Step 7 → Click Run Analysis
Step 8 → Review score, issues, and download the HTML report
```

### 3.2 CLI Path

```
# Analyze with HTML report output
python cli.py analyze Quadcopter_01.BIN --profile profiles/quadcopter.yaml --html

# View fleet overview
python cli.py fleet
```

### 3.3 Output Files

| Output         | Location                               | Description                      |
|----------------|----------------------------------------|----------------------------------|
| JSON report    | Same folder as input <code>.BIN</code> | Machine-readable full analysis   |
| HTML report    | <code>reports/</code> folder           | Self-contained, shareable report |
| Fleet database | <code>fleet.db</code>                  | SQLite, auto-created             |

---

## 4. Web UI — Analyze Tab

### 4.1 Upload Zone

Drag and drop a `.BIN` file onto the upload area, or click to browse. Only ArduPilot DataFlash `.BIN` files are accepted. Files must come directly from the flight controller SD card — not `.tlog` (telemetry) or `.log` (text) formats.

### 4.2 Drone Configuration

**Drone ID** — A text label that groups all flights from this airframe together in the fleet database. Use a stable, consistent name such as `UAV-001` or `QuadPlane-Survey`. Changing this ID creates a new drone entry in the database.

**Vehicle Type** — Selects the analysis context and profile:

| Category    | Sub-types             | Backend Profile                             |
|-------------|-----------------------|---------------------------------------------|
| Rotorcraft  | Quadcopter (4 motors) | <code>quadcopter</code>                     |
| Rotorcraft  | Hexacopter (6 motors) | <code>quadcopter</code> with motor override |
| Rotorcraft  | Octocopter (8 motors) | <code>quadcopter</code> with motor override |
| VTOL Hybrid | QuadPlane             | <code>quadplane</code>                      |
| Fixed-Wing  | Fixed-Wing            | <code>fixed_wing</code>                     |

Hexacopter and Octocopter sub-types automatically inject the correct motor count and channel assignments into the profile before analysis runs.

### 4.3 Profile Settings Panel

Click **Profile Settings** to expand the configuration panel. All thresholds are pre-filled from the selected vehicle profile. Adjust any field to customize the analysis for your specific airframe.

Fields are organized into sections:

#### Operational Parameters



| Field              | Unit    | Description                                                  |
|--------------------|---------|--------------------------------------------------------------|
| Expected Endurance | minutes | Planned flight duration — drives endurance deficit detection |
| Expected Range     | km      | Planned mission range                                        |
| Max Payload        | kg      | Payload capacity                                             |
| Max Speed          | m/s     | Operating speed limit — used for airspeed envelope           |
| Max Altitude       | m       | Operating altitude limit                                     |

### Battery

| Field                  | Unit    | Description                                |
|------------------------|---------|--------------------------------------------|
| Cell Count             | integer | Number of LiPo cells in series             |
| Capacity               | mAh     | Pack capacity                              |
| Full Voltage           | V       | Pack voltage at full charge                |
| Nominal Voltage        | V       | Pack nominal operating voltage             |
| Min Cell Voltage       | V       | Per-cell warning threshold                 |
| Critical Cell Voltage  | V       | Per-cell critical/failsafe threshold       |
| Max Continuous Current | A       | Rated maximum current                      |
| Cycle Lifespan         | cycles  | Expected pack cycle life before retirement |

### Motors (Lift)

| Field             | Unit    | Description                                                |
|-------------------|---------|------------------------------------------------------------|
| Motor Count       | integer | Number of lift motors (locked when Hex/Octo preset active) |
| Hover Throttle    | %       | Expected hover throttle — baseline for load analysis       |
| Warn Throttle     | %       | High throttle warning threshold                            |
| Critical Throttle | %       | High throttle critical threshold                           |
| Max PWM           | μs      | ESC maximum signal                                         |
| Min PWM           | μs      | ESC minimum signal                                         |

#### Pusher / Cruise Motor (VTOL only)

| Field             | Unit    | Description                                             |
|-------------------|---------|---------------------------------------------------------|
| Pusher Channel    | RCOU ch | ArduPilot output channel for cruise motor (default: C3) |
| Max PWM           | μs      | Pusher ESC maximum signal                               |
| Min PWM           | μs      | Pusher ESC minimum signal                               |
| Warn Throttle     | %       | Cruise motor high throttle warning                      |
| Critical Throttle | %       | Cruise motor saturation threshold                       |

#### Vibration

| Field | Unit | Description |
|-------|------|-------------|
|-------|------|-------------|

|                 |                  |                                     |
|-----------------|------------------|-------------------------------------|
| Warn Level      | m/s <sup>2</sup> | Vibration mean warning threshold    |
| Critical Level  | m/s <sup>2</sup> | Vibration mean critical threshold   |
| Clip Warn / min | events/min       | IMU clipping rate warning threshold |

## GPS / Navigation

| Field             | Unit  | Description                                         |
|-------------------|-------|-----------------------------------------------------|
| Min Satellites    | count | Minimum acceptable satellite count                  |
| Max HDOP          | —     | Maximum acceptable Horizontal Dilution of Precision |
| Min Fix Type      | —     | 2=2D, 3=3D, 4=DGPS, 6=RTK Fixed                     |
| Max Position Jump | m     | Maximum tolerated instantaneous GPS jump            |

## Maintenance Schedule

| Field                  | Unit          | Description                    |
|------------------------|---------------|--------------------------------|
| Motor Hours            | hours         | Lift motor inspection interval |
| Prop Cycles            | flight cycles | Propeller replacement interval |
| Battery Cycles         | charge cycles | Battery retirement threshold   |
| ESC Hours              | hours         | ESC inspection interval        |
| Frame Inspection Hours | hours         | Structural inspection interval |

## 4.4 Presets

Presets save the current profile settings in your browser's local storage. Use them to quickly switch between different airframe configurations without re-entering all fields.

- **Save Preset** — enter a name and save current field values
- **Load Preset** — select from dropdown to restore saved values
- **Delete Preset** — remove a saved configuration

Presets are browser-local and do not affect the server or YAML profile files.

## 4.5 Analysis Results

After clicking **Run Analysis**, the tool uploads the `.BIN`, processes all modules, and displays:

**Score Banner** — Overall score (0–100), letter grade (A–F), verdict text, and recommended action.

**Report Actions** — Three buttons: - **View HTML Report** — opens the generated report in a new tab - **Download HTML Report** — saves the self-contained HTML file - **Download JSON** — saves the raw machine-readable analysis data

**Module Radar Chart** — Spider chart showing each module's score. Modules that returned `available=false` are shown at 100 (not scored, no deduction).

**Module Score Cards** — One card per analysis module showing score, grade, and one-line summary.

**Issue List** — All detected issues with severity filter tabs (Critical / Warning / Info / All). Each issue shows: - Severity badge (red=Critical, orange=Warning, blue=Info) - Issue code (e.g. `BAT-002`) - Module source - Detailed message with observed value and threshold

---

## 5. Web UI — Fleet Dashboard

---

### 5.1 Overview Table

The overview table shows one row per drone (latest flight data). Columns include:

- Drone ID, Type, Total Flights
- Latest flight date and score
- Battery IR trend arrow
- Vibration level trend arrow
- Motor imbalance trend arrow
- Overall score trend arrow
- Maintenance alert indicator

Click the arrow on any drone row to expand the **Drone Detail Panel**.

**Search** — Type in the search box to filter drones by name in real time.

**Refresh** — Reload fleet data from the database.

**Clear All Fleet Data** — Permanently delete all flight records (confirmation required).

### 5.2 Drone Detail Panel

Expanding a drone shows:

- **Trend Graphs** — Time-series charts for key metrics across all flights. X-axis shows actual flight dates.
- **Flight History Table** — All analyzed flights with score, grade, issue counts, and duration.
- **Per-flight actions** — Delete a single flight record (trash icon, confirmation required).
- **Compare checkboxes** — Select two flights and click **Compare** for a side-by-side metric diff.
- **Export CSV** — Download full flight history as a spreadsheet.
- **Delete Drone** — Remove all flights for this drone (trash icon, confirmation required).

### 5.3 Maintenance Status

The **Maintenance** sub-view shows a grid of maintenance status indicators per drone, derived from accumulated flight hours and cycle counts stored in the fleet database. Each metric shows current usage vs. the inspection interval defined in the profile.

### 5.4 Compare View

Select two flights from the history table using the checkboxes, then click **Compare**. The compare table shows side-by-side values for all key metrics with color-coded delta indicators (green=improved, red=degraded).

---

## 6. Vehicle Profiles & YAML Reference

### 6.1 Built-In Profiles

Three ready-to-use profiles are included in the `profiles/` folder:

| File                                  | Vehicle Class     | Battery | Motors                        | Endurance |
|---------------------------------------|-------------------|---------|-------------------------------|-----------|
| <code>profiles/quadcopter.yaml</code> | Quadcopter        | 6S      | 4 (C1–C4)                     | 26 min    |
| <code>profiles/quadplane.yaml</code>  | VTOL<br>QuadPlane | 12S     | 4 lift (C5–C8) +<br>pusher C3 | 90 min    |
| <code>profiles/fixed_wing.yaml</code> | FixedWing         | 6S      | 1 (C3)                        | 60 min    |

Auto-detection profiles (used when no profile is specified) are in `profiles/defaults/`: - `copter_default.yaml` - `vtol_default.yaml` - `fw_default.yaml`

### 6.2 Creating a Custom Fleet Profile

```
# Start from the matching default
cp profiles/quadplane.yaml profiles/my_vtol_01.yaml
# Edit values for your specific airframe
# Then use it in CLI analysis
python cli.py analyze flight.bin --profile profiles/my_vtol_01.yaml
```

### 6.3 Full YAML Field Reference

```
id: "my-drone-id"          # Profile identifier (string)
name: "My Drone"           # Display name
type: vtol                  # multirotor | fixed_wing | vtol

# --- Operational ---
expected_endurance_min: 45 # Planned mission endurance (minutes)
expected_range_km: 30.0    # Planned mission range (km)
max_payload_kg: 1.5        # Payload capacity (kg)
max_speed_ms: 25.0         # Maximum operating speed (m/s)
max_altitude_m: 150.0      # Maximum operating altitude (m)

# --- Battery ---
battery:
  cell_count: 6             # LiPo series cell count (auto-detected from log)
  capacity_mah: 10000       # Pack capacity (mAh)
  nominal_voltage: 22.2     # Nominal pack voltage (V)
```

```

full_voltage: 25.2      # Full charge voltage (V)
min_cell_voltage: 3.5   # Per-cell warning threshold (V)
critical_cell_voltage: 3.3 # Per-cell critical threshold (V)
max_continuous_current_a: 60.0 # Rated max current (A)
cycles_lifespan: 300    # Expected pack cycle life

# --- Motors (Lift) ---
motors:
  count: 4              # Number of lift motors (auto-detected)
  channels: [5, 6, 7, 8] # RCOU channels (auto-detected)
  max_pwm: 2000         # ESC max signal (µs)
  min_pwm: 1000         # ESC min signal (µs)
  hover_throttle_pct: 55 # Expected hover throttle (%)
  high_throttle_warn_pct: 80 # Warning threshold (%)
  high_throttle_critical_pct: 92 # Critical threshold (%)

# --- Pusher / Cruise Motor (VTOL only) ---
pusher_motor:
  channel: 3            # RCOU output channel
  max_pwm: 2000
  min_pwm: 1000
  high_throttle_warn_pct: 80
  high_throttle_critical_pct: 92

# --- Vibration ---
vibration:
  warn_threshold: 15.0   # Mean vibration warning (m/s²)
  critical_threshold: 30.0 # Mean vibration critical (m/s²)
  clip_warn_per_min: 5   # IMU clip event rate warning

# --- GPS / Navigation ---
gps:
  min_satellites: 8
  max_hdop: 1.5
  min_fix_type: 3        # 3 = 3D Fix
  max_position_jump_m: 15.0

# --- Maintenance Schedule ---
maintenance:
  motor_hours: 200
  prop_cycles: 150
  battery_cycles: 300
  esc_hours: 200
  frame_inspection_hours: 50

# --- Scoring Weights (must sum to 1.0) ---
scoring_weights:
  battery: 0.09
  motors: 0.07
  flight_overview: 0.10
  sensors: 0.09
  # ... (see Section 8 for full tables)

# --- Threshold Overrides (optional, advanced) ---

```



```
# thresholds:  
#   ctl_rms_warn_deg: 3.0  
#   mot_sat_warn_pct: 5.0
```

## 6.4 Auto-Detection Behavior

When no profile is specified (Web UI always sends one; CLI defaults to `quadcopter.yaml`): - **Cell count** is auto-detected from log voltage at runtime and overrides the profile value - **Motor channels** are auto-detected from RCOU idle threshold analysis - **Firmware type** is auto-detected: XKF4 or TECS messages → ArduPlane; otherwise → ArduCopter

---

## 7. Analysis Modules Reference

Every module returns the same standard structure:

```
{
  "score": 95.0,
  "grade": "A",
  "available": true,
  "summary": "One-line interpretation",
  "issues": [ { "severity": "warning", "code": "BAT-002", "message": "..." } ],
  "metrics": { "key": "value" }
}
```

When `available` is `false`, the module is excluded from scoring and its weight is redistributed to the remaining available modules.

### 7.1 Flight Overview (`flight_overview`)

**Purpose:** Summarize the overall flight — duration, phases, GPS coverage, altitude range, and endurance against plan.

**Required Data:** GPS, MODE, ATT or BARO messages

**Key Metrics:** - Total armed duration (seconds) - Flight phases detected (e.g. AUTO, LOITER, GUIDED) - Maximum AGL altitude (m) - Ground track distance (km) - Battery remaining at landing (%)

**Issue Codes:**

| Code    | Severity | Meaning                                      | Action                                       |
|---------|----------|----------------------------------------------|----------------------------------------------|
| OVR-001 | Info     | Flight summary: duration, distance, altitude | —                                            |
| OVR-002 | Warning  | Endurance significantly below expected       | Review payload, battery, or profile settings |
| OVR-003 | Warning  | Early landing with low battery remaining     | Check endurance planning and RTL trigger     |

|         |      |                       |   |
|---------|------|-----------------------|---|
| OVR-004 | Info | Flight phases summary | — |
|---------|------|-----------------------|---|

## 7.2 Battery (battery)

**Purpose:** Analyze voltage curve, current draw, per-cell health, internal resistance (IR), and consumption.

**Required Data:** BAT messages (voltage, current, remaining)

**Key Metrics:** - Starting cell voltage (V) - Landing cell voltage (V) - Minimum cell voltage during flight (V) - Maximum current drawn (A) - Energy consumed (Wh) - Estimated battery internal resistance (mOhm)

**Issue Codes:**

| Code    | Severity | Meaning                                                       | Action                                            |
|---------|----------|---------------------------------------------------------------|---------------------------------------------------|
| BAT-000 | Info     | Battery summary (voltage, current, capacity)                  | —                                                 |
| BAT-001 | Warning  | Low starting voltage (battery not fully charged)              | Charge to full before flight                      |
| BAT-002 | Critical | Cell voltage dropped below critical threshold during flight   | Land immediately; check battery health            |
| BAT-003 | Warning  | Cell voltage dropped below warning threshold                  | Reduce flight aggressiveness or increase capacity |
| BAT-004 | Critical | Voltage drop spike (possible loose connector or cell failure) | Inspect battery and connector                     |
| BAT-005 | Warning  | High current draw (above rated max)                           | Check motor load, reduce throttle demands         |

|         |          |                                                  |                                            |
|---------|----------|--------------------------------------------------|--------------------------------------------|
| BAT-006 | Warning  | Low battery remaining at landing                 | Review endurance planning                  |
| BAT-007 | Critical | Battery critically depleted (very low remaining) | Immediate battery retirement check         |
| BAT-008 | Warning  | High estimated internal resistance               | Battery may be aging; consider replacement |
| BAT-009 | Warning  | High IR variance across cells (cell imbalance)   | Balance charge; inspect individual cells   |
| BAT-010 | Info     | Multiple battery monitors detected; primary used | —                                          |

## 7.3 Motors (motors)

**Purpose:** Analyze throttle loading, motor symmetry (for multirotors), saturation events, oscillation, and dead motor detection.

**Required Data:** RCOU messages (motor PWM outputs)

**Key Metrics:** - Per-motor mean and max throttle (%) - Motor imbalance (max deviation from mean) - Throttle saturation percentage - Pusher motor mean/max throttle (VTOL only)

**Issue Codes:**

| Code    | Severity | Meaning                       | Action                             |
|---------|----------|-------------------------------|------------------------------------|
| MOT-000 | Info     | Motor summary                 | —                                  |
| MOT-001 | Warning  | High motor symmetry imbalance | Check propellers and motor balance |

|         |          |                                                                |                                                 |
|---------|----------|----------------------------------------------------------------|-------------------------------------------------|
| MOT-002 | Critical | Very high motor imbalance (possible dead motor or broken prop) | Inspect all motors and propellers before flight |
| MOT-003 | Warning  | Sustained high throttle (above warn threshold)                 | Check payload weight; motor/battery sizing      |
| MOT-004 | Critical | Sustained critical throttle (near saturation)                  | Ground aircraft; review power system            |
| MOT-005 | Warning  | Motor throttle saturation events detected                      | Reduce aggressiveness; check airframe balance   |
| MOT-006 | Warning  | High fixed-wing throttle (above warn)                          | Review propulsion sizing or headwind condition  |
| MOT-007 | Info     | Motor channel summary                                          | —                                               |
| MOT-008 | Critical | Fixed-wing critical throttle (possible engine struggling)      | Inspect motor and propeller                     |
| MOT-009 | Critical | Possible dead motor detected (near-zero output)                | Inspect motor and ESC before next flight        |
| MOT-010 | Warning  | Pusher/cruise motor high throttle (above warn)                 | Review cruise speed and propulsion sizing       |
| MOT-011 | Critical | Pusher/cruise motor saturation (near max output)               | Inspect pusher motor and reduce cruise demands  |

## 7.4 Vibration (vibration)

**Purpose:** Analyze IMU vibration levels, clipping events, and vibration trends that indicate prop damage, motor bearing failure, or mounting issues.

**Required Data:** VIBE messages, optionally IMU messages for FFT

**Key Metrics:** - Mean vibration per axis (m/s<sup>2</sup>) - Peak vibration (m/s<sup>2</sup>) - IMU clipping events per minute - Vibration trend (rising/stable/falling)

**Issue Codes:**

| Code    | Severity | Meaning                                   | Action                                         |
|---------|----------|-------------------------------------------|------------------------------------------------|
| VIB-000 | Info     | Vibration summary                         | —                                              |
| VIB-001 | Warning  | Vibration level above warning threshold   | Check propeller balance and motor mounts       |
| VIB-002 | Critical | Vibration level above critical threshold  | Ground aircraft; inspect propellers and motors |
| VIB-003 | Warning  | IMU clipping events detected              | Reduce vibration; check mounting foam          |
| VIB-004 | Warning  | IMU clipping rate above warning threshold | Investigate vibration source                   |
| VIB-005 | Warning  | Rising vibration trend across flight      | Monitor for progressive prop/bearing damage    |
| VIB-006 | Info     | FFT analysis summary                      | —                                              |

## 7.5 Sensors (sensors)

**Purpose:** Evaluate GPS fix quality, EKF health, IMU consistency, magnetometer noise, and barometer stability.

**Required Data:** GPS, XKF4, IMU, MAG, BARO messages

**Issue Codes:**

| Code    | Severity | Meaning                                  | Action                                               |
|---------|----------|------------------------------------------|------------------------------------------------------|
| SEN-001 | Info     | GPS summary                              | —                                                    |
| SEN-002 | Warning  | Low satellite count                      | Improve sky view;<br>check antenna<br>placement      |
| SEN-003 | Critical | GPS fix lost or<br>insufficient fix type | Do not fly<br>autonomously until<br>fix restored     |
| SEN-010 | Info     | EKF summary                              | —                                                    |
| SEN-011 | Warning  | EKF velocity<br>variance elevated        | Check GPS and IMU                                    |
| SEN-012 | Critical | EKF health critical                      | Do not fly; investigate<br>sensor fusion             |
| SEN-013 | Warning  | High HDOP                                | Check GPS antenna;<br>move away from<br>obstructions |
| SEN-014 | Warning  | GPS position jump<br>detected            | Check for RF<br>interference                         |
| SEN-020 | Info     | IMU summary                              | —                                                    |
| SEN-021 | Warning  | IMU temperature<br>variance elevated     | Check IMU<br>heating/cooling                         |
| SEN-022 | Warning  | Multiple IMU<br>disagreement             | Check IMU isolation<br>and calibration               |
| SEN-030 | Info     | Magnetometer<br>summary                  | —                                                    |

|         |         |                                   |                                             |
|---------|---------|-----------------------------------|---------------------------------------------|
| SEN-031 | Warning | Magnetic field strength deviation | Recalibrate compass; check for interference |
| SEN-032 | Warning | Magnetometer inconsistency        | Inspect for metallic interference sources   |
| SEN-040 | Info    | Barometer summary                 | —                                           |
| SEN-041 | Warning | Barometer noise elevated          | Check for airflow around barometer          |
| SEN-050 | Info    | Sensor health summary             | —                                           |
| SEN-051 | Warning | Multiple sensor warnings detected | Investigate sensor suite                    |

## 7.6 Control (control)

**Purpose:** Measure how accurately the autopilot tracked desired attitude (roll, pitch, yaw), detect PID oscillation, and identify large attitude errors.

**Required Data:** ATT messages (desired vs actual roll/pitch/yaw)

**Key Metrics:** - Roll/Pitch RMS tracking error (degrees) - Yaw tracking error (degrees) - Percentage of flight with large attitude errors - Oscillation frequency and amplitude

**Issue Codes:**

| Code    | Severity | Meaning                                     | Action                                        |
|---------|----------|---------------------------------------------|-----------------------------------------------|
| CTL-001 | Warning  | Roll/Pitch RMS tracking error above warning | Review PID tune; check for mechanical play    |
| CTL-002 | Critical | Roll/Pitch RMS tracking error critical      | Ground aircraft; retune PID or inspect servos |



|         |         |                                                          |                                                |
|---------|---------|----------------------------------------------------------|------------------------------------------------|
| CTL-003 | Warning | Large attitude errors detected (> threshold % of flight) | Check airframe integrity and PID stability     |
| CTL-004 | Warning | Yaw tracking deviation elevated                          | Check compass calibration and yaw PID          |
| CTL-005 | Warning | Oscillation detected in attitude control                 | Reduce P-gain; check for resonance frequencies |

## 7.7 Efficiency (efficiency)

**Purpose:** Compute energy efficiency (Wh/km), average power consumption, and compare against profile baseline.

**Required Data:** BAT messages + GPS messages

**Key Metrics:** - Energy consumed (Wh) - Total distance (km) - Efficiency (Wh/km) - Average power (W) - Deviation from expected efficiency

**Issue Codes:**

| Code    | Severity | Meaning                                 | Action                                          |
|---------|----------|-----------------------------------------|-------------------------------------------------|
| EFF-001 | Warning  | Efficiency significantly below expected | Check payload weight, wind conditions, PID tune |
| EFF-002 | Info     | Efficiency summary                      | —                                               |
| EFF-003 | Warning  | High average power consumption          | Review propulsion sizing and flight profile     |

## 7.8 RC Link (rc\_link)

**Purpose:** Detect RC signal dropouts, failsafe activations, and RSSI quality degradation.

**Required Data:** RCIN, RSSI, ERR messages

#### Issue Codes:

| Code    | Severity | Meaning                             | Action                                         |
|---------|----------|-------------------------------------|------------------------------------------------|
| RCL-001 | Warning  | RC signal dropout detected          | Check transmitter battery; reduce interference |
| RCL-002 | Critical | RC failsafe activated during flight | Investigate RC range and antenna orientation   |
| RCL-003 | Warning  | Low RC signal RSSI                  | Move closer; check antenna; reduce obstacles   |
| RCL-005 | Info     | RC link summary                     | —                                              |

**Note:** RC link module may return `available=false` for fully autonomous missions where RC inputs are not logged.

## 7.9 Landing & Wind (`landing_wind`)

**Purpose:** Assess landing quality (bounce, hard landing, lateral drift) and estimate wind from GPS/attitude data.

**Required Data:** GPS, ATT, BARO messages near landing

#### Issue Codes:

| Code    | Severity | Meaning                                            | Action                                      |
|---------|----------|----------------------------------------------------|---------------------------------------------|
| LND-001 | Info     | Landing summary                                    | —                                           |
| LND-002 | Warning  | Hard landing detected (rapid descent at touchdown) | Adjust landing approach; check landing gear |
| LND-003 | Warning  | Bounce detected at landing                         | Tune landing PID; reduce descent rate       |

|         |         |                                          |                                                |
|---------|---------|------------------------------------------|------------------------------------------------|
| LND-004 | Warning | High lateral drift at landing            | Check crosswind limits; improve approach angle |
| LND-005 | Warning | High estimated wind speed during landing | Verify wind limits for operation               |
| WND-001 | Info    | Wind estimate summary                    | —                                              |
| WND-002 | Warning | Estimated wind above operational limit   | Review mission go/no-go criteria               |
| WND-003 | Info    | Wind direction noted                     | —                                              |

### 7.10 Flight Controller Health (fc\_health)

**Purpose:** Monitor flight controller power rails (VCC, servo bus), detect brownouts, and flag ArduPilot error events.

**Required Data:** POWR, ERR messages

**Key Metrics:** - VCC rail mean and min (V) - Servo rail mean (V) - Brownout events (VCC change > 200 mV)  
- Firmware error events (ERR messages)

**Issue Codes:**

| Code    | Severity | Meaning                                                | Action                                               |
|---------|----------|--------------------------------------------------------|------------------------------------------------------|
| FCH-001 | Info     | FC power summary                                       | —                                                    |
| FCH-002 | Warning  | VCC rail voltage low                                   | Check FC power supply and cable connections          |
| FCH-003 | Critical | VCC brownout detected (VCC dropped > 200 mV in flight) | Inspect power distribution; replace BEC if necessary |

|         |          |                                    |                                             |
|---------|----------|------------------------------------|---------------------------------------------|
| FCH-004 | Warning  | Servo rail voltage anomaly         | Check servo BEC                             |
| FCH-005 | Warning  | ArduPilot error event logged       | Review ERR log for subsystem and error code |
| FCH-006 | Critical | Multiple ArduPilot errors detected | Investigate firmware and hardware           |
| FCH-007 | Info     | No brownout events detected        | —                                           |
| FCH-008 | Warning  | Pre-arm power event noted          | Check power system before arming            |

### 7.11 VTOL Transition (vtol\_transition)

**Purpose:** Evaluate the quality of transitions between hover (Q-mode) and fixed-wing cruise, and back.

**Required Data:** MODE, ATT, GPS, BARO messages (VTOL logs only)

**Availability:** Returns `available=false` for non-VTOL vehicle types.

**Key Metrics:** - Forward transition duration (s) - Altitude loss/gain during transition (m) - Attitude spike during transition (degrees) - Back transition quality

**Issue Codes:**

| Code    | Severity | Meaning                                  | Action                                               |
|---------|----------|------------------------------------------|------------------------------------------------------|
| VTR-001 | Info     | VTOL transition summary                  | —                                                    |
| VTR-002 | Warning  | Altitude loss during forward transition  | Review ARSPD <i>FBW</i> MIN and transition speed     |
| VTR-003 | Critical | Critical altitude loss during transition | Adjust transition parameters; verify airspeed sensor |

|         |         |                                        |                                                     |
|---------|---------|----------------------------------------|-----------------------------------------------------|
| VTR-004 | Warning | Large attitude spike during transition | Tune transition <i>QTILT</i> or <i>QTRANS_DECEL</i> |
| VTR-005 | Warning | Slow forward transition                | Check transition speed and motor tilt rate          |
| VTR-006 | Warning | Back transition anomaly                | Review Q_ASSIST and back-transition parameters      |
| VTR-007 | Info    | Transition event details               | —                                                   |

## 7.12 Mission Execution (mission)

**Purpose:** Verify mission waypoint completion, measure cross-track error (deviation from planned path), and detect missed waypoints.

**Required Data:** CMD, GPS, MODE messages

**Availability:** Returns `available=false` when no CMD data, no AUTO mode segments, or fewer than 2 nav waypoints are detected.

**Key Metrics:** - Total waypoints planned vs executed - Waypoints missed - Mean cross-track error (m) - Max cross-track error (m) - Per-leg statistics

**Issue Codes:**

| Code    | Severity | Meaning                          | Action                                          |
|---------|----------|----------------------------------|-------------------------------------------------|
| MSN-000 | Info     | Mission summary                  | —                                               |
| MSN-001 | Warning  | Waypoints missed or not executed | Check CMD upload; verify AUTO mode triggers     |
| MSN-002 | Critical | Multiple waypoints not executed  | Investigate GPS, RC failsafe, or GCS connection |

|         |          |                                           |                                               |
|---------|----------|-------------------------------------------|-----------------------------------------------|
| MSN-003 | Warning  | Cross-track error above warning threshold | Tune L1 navigation controller                 |
| MSN-004 | Critical | Cross-track error critically high         | Inspect navigation system; check GPS and wind |
| MSN-005 | Info     | Per-leg cross-track summary               | —                                             |

### 7.13 Airspeed Health (airspeed)

**Purpose:** Analyze stall risk and overspeed events using either a dedicated ARSP sensor or GPS groundspeed as a proxy.

**Required Data:** ARSP messages (preferred) or GPS messages

**Availability:** Returns `available=false` for multirotor (no airspeed envelope). Available for fixed-wing and VTOL cruise segments only.

**Data Sources (in priority order):** 1. **ARSP sensor** — used if present in log (requires `LOG_BITMASK` bit for airspeed logging) 2. **GPS groundspeed** — proxy fallback when no ARSP data

Speed thresholds derived from profile: - Stall proxy = `max_speed_ms × 0.40` (40% of cruise speed) - Overspeed threshold = `max_speed_ms × 1.25` (125% of cruise speed)

#### Issue Codes:

| Code    | Severity         | Meaning                                                      | Action                                                   |
|---------|------------------|--------------------------------------------------------------|----------------------------------------------------------|
| ASP-001 | Info             | Airspeed source summary (ARSP sensor or GPS proxy)           | —                                                        |
| ASP-002 | Critical/Warning | Stall risk: > 5% of cruise time below minimum airspeed proxy | Check ARSPD <code>FBW</code> MIN; review approach speeds |

|         |         |                                                            |                                                              |
|---------|---------|------------------------------------------------------------|--------------------------------------------------------------|
| ASP-003 | Warning | Overspeed: > 2% of cruise time above maximum airspeed      | Check ARSPD <code>FBW</code> MAX; review dive speeds         |
| ASP-004 | Warning | High airspeed variability (turbulence or throttle hunting) | Review TECST <code>THR</code> DAMP or L1 controller settings |

**Note on ASP-002 gusty conditions:** When GPS groundspeed variability is very high (std/mean > 0.50), indicating significant wind, ASP-002 is downgraded from Critical to Warning with reduced score penalty. Install a dedicated airspeed sensor for accurate stall protection.

### 7.14 PID Tuning Assessment (pid\_tuning)

**Purpose:** Evaluate PID controller health — I-term saturation, D-term noise, and rate tracking error.

**Required Data:** PIDR, PIDP messages

**Availability:** Returns `available=false` when PIDR/PIDP messages are absent. Enable by setting LOG\_BITMASK bit 6 in ArduPilot parameters.

**Issue Codes:**

| Code    | Severity | Meaning                                  | Action                                         |
|---------|----------|------------------------------------------|------------------------------------------------|
| PID-001 | Info     | PID summary                              | —                                              |
| PID-002 | Warning  | High I-term saturation                   | Reduce I-gain; check for mechanical binding    |
| PID-003 | Warning  | High D-term noise                        | Check vibration; reduce D-gain or add D-filter |
| PID-004 | Warning  | Poor rate tracking (high rate error RMS) | Retune P-gain for rate loop                    |
| PID-005 | Info     | PID quality good                         | —                                              |

## 7.15 Power Rail Health (power\_rail)

**Purpose:** Monitor main power bus stability, detect brownout events, measure voltage sag under load, and assess bus noise.

**Required Data:** BAT messages (voltage vs current), POWR messages

**Availability:** Returns `available=false` when BAT messages are absent or fewer than 10 armed samples exist.

**Issue Codes:**

| Code    | Severity | Meaning                                              | Action                                                  |
|---------|----------|------------------------------------------------------|---------------------------------------------------------|
| PWR-001 | Critical | Brownout detected: VCC changed by > 200 mV in flight | Inspect power distribution and BEC                      |
| PWR-002 | Warning  | Peak voltage sag during high-current events          | Increase battery C-rating or reduce load                |
| PWR-003 | Warning  | Sustained voltage sag under load                     | Inspect wiring resistance; upgrade to lower-ESR battery |
| PWR-004 | Warning  | High bus voltage noise (CV > threshold)              | Check capacitor; inspect ESC and wiring                 |

## 7.16 Telemetry Link Quality (telemetry)

**Purpose:** Assess ground-to-air telemetry radio quality: RSSI, SNR, packet errors, and TxBuf congestion.

**Required Data:** RADIO messages (SiK / RFD900 radio)

**Availability:** Returns `available=false` when RADIO messages are absent. Expected when no telemetry radio is connected or radio statistics logging is not enabled.

**Issue Codes:**

| Code | Severity | Meaning | Action |
|------|----------|---------|--------|
|------|----------|---------|--------|



|         |         |                                         |                                             |
|---------|---------|-----------------------------------------|---------------------------------------------|
| TLM-001 | Info    | Telemetry link summary                  | —                                           |
| TLM-002 | Warning | RSSI below acceptable level             | Reduce range; improve antenna orientation   |
| TLM-003 | Warning | Low SNR (signal-to-noise ratio)         | Check for RF interference; change frequency |
| TLM-004 | Warning | TxBuf congestion (bandwidth saturation) | Reduce telemetry stream rates in GCS        |
| TLM-005 | Warning | High RxErrors rate                      | Check cable and radio hardware              |

## 7.17 ESC Telemetry Health (`esc_telemetry`)

**Purpose:** Monitor per-motor ESC board temperature, motor winding temperature, current imbalance, RPM imbalance, and ESC error counts.

**Required Data:** ESC messages (BLHeli32 / AM32 passthrough telemetry)

**Availability:** Returns `available=false` when ESC messages are absent. Enable by configuring BLHeli32 or AM32 ESC telemetry passthrough in ArduPilot parameters.

**Thresholds:** - ESC board temp: warn 60°C, critical 80°C - Motor winding temp: warn 70°C, critical 100°C - Current imbalance: warn 20%, critical 40% - RPM imbalance: warn 15%, critical 30%

### Issue Codes:

| Code    | Severity         | Meaning                        | Action                             |
|---------|------------------|--------------------------------|------------------------------------|
| ESC-001 | Warning/Critical | ESC board temperature high     | Improve ESC cooling; check airflow |
| ESC-002 | Warning/Critical | Motor winding temperature high | Check motor load and cooling       |

|         |                  |                                      |                                                  |
|---------|------------------|--------------------------------------|--------------------------------------------------|
| ESC-003 | Warning/Critical | ESC current imbalance between motors | Check propellers; inspect for motor drag         |
| ESC-004 | Warning/Critical | ESC RPM imbalance between motors     | Inspect for partial motor failure or prop damage |

## 7.18 Altitude Control (`altitude_control`)

**Purpose:** Measure altitude tracking accuracy (commanded vs actual), climb-rate tracking, and altitude hold stability during loiter.

**Required Data:** CTUN messages (ArduCopter), NTUN messages (ArduPlane/VTOL)

**Key Metrics:** - Altitude tracking RMS error (m) - Peak altitude deviation (m) - Climb-rate RMS error (m/s) - Altitude hold standard deviation during loiter/alt-hold

**Issue Codes:**

| Code    | Severity         | Meaning                                    | Action                                                  |
|---------|------------------|--------------------------------------------|---------------------------------------------------------|
| CTN-001 | Warning          | Altitude tracking RMS above warning        | Tune TECS or altitude PID                               |
| CTN-002 | Critical         | Altitude tracking RMS above critical       | Inspect altitude sensor; tune controller                |
| CTN-003 | Warning/Critical | Peak altitude deviation too large          | Check for disturbances; verify alt sensor               |
| CTN-004 | Warning          | Poor altitude hold stability during loiter | Tune <code>TECSHDEMTCONST</code> or altitude controller |

**Per-segment settling:** The first 30 s (multirotor) or 30–60 s (fixed-wing/VTOL) of each AUTO segment is excluded from analysis to avoid penalizing normal controller convergence on mode entry.

## 7.19 Firmware Error Events (fc\_health — ERR subsystem)

ArduPilot logs [ERR](#) messages for firmware-level fault events. These appear as:

```
ERR-XX (where XX = subsystem ID)
```

Common subsystems: - ERR-02: Compass - ERR-03: Optical Flow - ERR-04: Failsafe (throttle) - ERR-05: Failsafe (battery) - ERR-06: Failsafe (GPS) - ERR-08: Parachute - ERR-09: EKF Primary Changed

ERR messages are included in [fc\\_health](#) issues with code `ERR-{subsystem_id:02d}`.

---

## 8. Scoring & Grading System

### 8.1 Grade Bands

| Score  | Grade | Verdict   | Recommended Action                                  |
|--------|-------|-----------|-----------------------------------------------------|
| 90–100 | A     | EXCELLENT | Ready for next mission                              |
| 75–89  | B     | GOOD      | Monitor; schedule routine check                     |
| 60–74  | C     | FAIR      | Investigate warnings before next flight             |
| 40–59  | D     | POOR      | Schedule maintenance; do not fly without inspection |
| 0–39   | F     | CRITICAL  | Ground aircraft; immediate maintenance required     |

### 8.2 Weighted Score Calculation

```
weighted_score = sum(module_score[i] × weight[i] for available modules)
                  / sum(weight[i] for available modules)
```

When a module returns `available=false`, its weight is removed and the remaining weights are renormalized to sum to 1.0. Modules that return `available=false` do NOT reduce the score.

### 8.3 Scoring Weights by Vehicle Type

Quadcopter (17 active modules):

| Module          | Weight |
|-----------------|--------|
| flight_overview | 0.10   |

|                  |      |
|------------------|------|
| battery          | 0.11 |
| sensors          | 0.09 |
| motors           | 0.08 |
| landing_wind     | 0.06 |
| control          | 0.06 |
| mission          | 0.06 |
| rc_link          | 0.05 |
| pid_tuning       | 0.05 |
| vibration        | 0.06 |
| efficiency       | 0.04 |
| fc_health        | 0.04 |
| airspeed         | 0.04 |
| power_rail       | 0.04 |
| telemetry        | 0.04 |
| esc_telemetry    | 0.04 |
| altitude_control | 0.04 |

**VTOL QuadPlane (18 active modules):**

| Module          | Weight |
|-----------------|--------|
| flight_overview | 0.10   |

|                  |      |
|------------------|------|
| battery          | 0.09 |
| sensors          | 0.09 |
| vtol_transition  | 0.06 |
| motors           | 0.07 |
| control          | 0.06 |
| mission          | 0.05 |
| airspeed         | 0.05 |
| pid_tuning       | 0.05 |
| rc_link          | 0.05 |
| landing_wind     | 0.05 |
| vibration        | 0.04 |
| efficiency       | 0.04 |
| fc_health        | 0.04 |
| power_rail       | 0.04 |
| telemetry        | 0.04 |
| esc_telemetry    | 0.04 |
| altitude_control | 0.04 |

#### Fixed-Wing (17 active modules):

| Module | Weight |
|--------|--------|
|--------|--------|

|                  |      |
|------------------|------|
| flight_overview  | 0.12 |
| battery          | 0.12 |
| sensors          | 0.09 |
| control          | 0.06 |
| landing_wind     | 0.06 |
| airspeed         | 0.05 |
| mission          | 0.05 |
| motors           | 0.05 |
| pid_tuning       | 0.05 |
| efficiency       | 0.05 |
| rc_link          | 0.05 |
| vibration        | 0.05 |
| fc_health        | 0.04 |
| power_rail       | 0.04 |
| telemetry        | 0.04 |
| esc_telemetry    | 0.04 |
| altitude_control | 0.04 |

## 8.4 Issue Severity Impact

Scoring deductions are applied within each module independently. The most severe issues have the largest penalty. Representative penalty examples:

| Severity Pattern | Typical Score Impact per Module |
|------------------|---------------------------------|
| Single Critical  | –20 to –30 points within module |
| Single Warning   | –5 to –15 points within module  |
| Info only        | 0 points (informational)        |

Module scores are clamped to the range 0–100 before aggregation.

---



## 9. Fleet & Trend Monitoring

### 9.1 Fleet Database Structure

All analyzed flights are stored in `fleet.db` (SQLite, auto-created). Records are keyed by `(drone_name, log_file)` — analyzing the same log file for the same drone again overwrites the existing record (upsert).

Data stored per flight: - Identity: drone name, type, log filename, flight date, analysis timestamp - Summary: overall score, grade, issue counts - All 18 module scores - Key metrics: battery IR, vibration mean, motor imbalance, efficiency, landing quality

### 9.2 Trend Detection

Trends are computed when a drone has **3 or more flights** in the database. A linear regression is fitted across the last N flights for each tracked metric.

Tracked metrics for trend alerts:

| Metric                      | Alert Direction | Alert Threshold                          |
|-----------------------------|-----------------|------------------------------------------|
| Overall score               | Declining       | Slope < -2 per flight                    |
| Battery internal resistance | Rising          | Slope > +1 mOhm per flight               |
| Vibration mean              | Rising          | Slope > +0.5 m/s <sup>2</sup> per flight |
| Motor imbalance             | Rising          | Slope > +1% per flight                   |

### 9.3 Trend Symbols

| Symbol | Direction | Meaning                            |
|--------|-----------|------------------------------------|
| ↑      | Rising    | Metric is increasing               |
| ↓      | Falling   | Metric is decreasing               |
| →      | Stable    | Metric is stable                   |
| ■      | Alert     | Trend has exceeded alert threshold |

## 9.4 Fleet API Endpoints

| Method | Endpoint                           | Purpose                            |
|--------|------------------------------------|------------------------------------|
| GET    | /api/fleet                         | Overview table (one row per drone) |
| GET    | /api/fleet/{drone_id}/history      | Full history + trends + alerts     |
| GET    | /api/fleet/{drone_id}/export       | CSV download (up to 500 flights)   |
| DELETE | /api/fleet/{drone_id}/flights/{id} | Delete one flight record           |
| DELETE | /api/fleet/{drone_id}              | Delete all flights for a drone     |
| DELETE | /api/fleet                         | Clear entire fleet database        |

## 10. CLI Reference

---

### 10.1 Commands

```
# Analyze a flight log
python cli.py analyze <LOG_FILE> [OPTIONS]

# View fleet summary
python cli.py fleet [OPTIONS]
```

### 10.2 Analyze Command Options

| Flag                     | Short           | Default                              | Description                            |
|--------------------------|-----------------|--------------------------------------|----------------------------------------|
| <code>--profile</code>   | <code>-p</code> | auto-detect                          | Path to YAML profile file              |
| <code>--output</code>    | <code>-o</code> | <code>&lt;log&gt;_report.json</code> | JSON output file path                  |
| <code>--html</code>      | —               | off                                  | Also generate HTML report              |
| <code>--no-report</code> | —               | off                                  | Skip JSON write; console output only   |
| <code>--drone-id</code>  | —               | profile name                         | Override drone name for fleet grouping |
| <code>--verbose</code>   | <code>-v</code> | off                                  | Enable debug logging                   |

### 10.3 Fleet Command Options

| Flag                 | Short           | Default | Description                         |
|----------------------|-----------------|---------|-------------------------------------|
| <code>--drone</code> | <code>-d</code> | —       | Show history + trends for one drone |

---

--all

—

off

Show all flight  
records flat

---

## 10.4 Example Commands

```
# Analyze each vehicle type with matching profile
python cli.py analyze Quadcopter_01.BIN --profile profiles/quadcopter.yaml --html
python cli.py analyze Quad_plane_01.BIN --profile profiles/quadplane.yaml --html
python cli.py analyze Fixed_wing_01.BIN --profile profiles/fixed_wing.yaml --html

# Custom drone ID for fleet grouping
python cli.py analyze flight.bin --profile profiles/quadcopter.yaml \
    --drone-id "UAV-Survey-01" --html

# Save JSON to specific location
python cli.py analyze flight.bin --profile profiles/quadplane.yaml \
    --output reports/mission_42.json --html

# Fleet overview
python cli.py fleet

# Single drone trend view
python cli.py fleet --drone "UAV-Survey-01"

# All records flat list
python cli.py fleet --all
```

---

## 11. API Reference

### 11.1 Base URL

```
http://localhost:5000
```

No authentication is required for local use. For shared/network deployments, add reverse-proxy authentication.

### 11.2 Endpoints Summary

| Method | Endpoint                           | Description                       |
|--------|------------------------------------|-----------------------------------|
| GET    | /                                  | Serve web frontend                |
| GET    | /api/profiles                      | List available vehicle types      |
| GET    | /api/profiles/{vehicle_type}       | Get editable profile payload      |
| POST   | /api/analyze                       | Upload .BIN and run analysis      |
| GET    | /api/fleet                         | Fleet overview (latest per drone) |
| GET    | /api/fleet/drone-ids               | List distinct drone IDs           |
| GET    | /api/fleet/{drone_id}/history      | Drone history + trends + alerts   |
| GET    | /api/fleet/{drone_id}/export       | Download history as CSV           |
| DELETE | /api/fleet/{drone_id}/flights/{id} | Delete one flight                 |
| DELETE | /api/fleet/{drone_id}              | Delete all flights for a drone    |
| DELETE | /api/fleet                         | Clear entire fleet database       |

### 11.3 POST /api/analyze

**Request:** multipart/form-data

| Field             | Type        | Required | Description                               |
|-------------------|-------------|----------|-------------------------------------------|
| log_file          | file        | Yes      | .BIN flight log file                      |
| drone_id          | string      | Yes      | Drone identifier                          |
| vehicle_type      | string      | Yes      | quadcopter /<br>quadplane /<br>fixed_wing |
| profile_overrides | JSON string | No       | Runtime profile field<br>overrides        |

#### Example — cURL:

```
curl -X POST "http://localhost:5000/api/analyze" \  
-F "log_file=@Quadcopter_01.BIN" \  
-F "drone_id=UAV-001" \  
-F "vehicle_type=quadcopter" \  
-F 'profile_overrides={"battery":{"cell_count":6}}'
```

#### Example — Python:

```
import json, requests  
  
r = requests.post(  
    "http://localhost:5000/api/analyze",  
    files={"log_file": open("flight.bin", "rb")},  
    data={  
        "drone_id": "UAV-001",  
        "vehicle_type": "quadplane",  
        "profile_overrides": json.dumps({  
            "expected_endurance_min": 45,  
            "battery": {"cell_count": 12, "capacity_mah": 22000}  
        })  
    },  
    timeout=300  
)  
report = r.json()  
print(report["flight_summary"]["overall_score"])
```

#### Response structure:

```
{
  "meta": {
    "analyzer_version": "1.0.0",
    "generated_at": "2026-04-10T08:00:00Z",
    "log_file": "flight.BIN",
    "drone_name": "UAV-001",
    "drone_type": "vtol"
  },
  "flight_summary": {
    "overall_score": 94.8,
    "grade": "A",
    "verdict": "EXCELLENT",
    "action": "Ready for next mission",
    "issue_summary": { "critical": 0, "warning": 3, "info": 6, "total": 9 },
    "module_scores": { "battery": { "score": 96.0, "grade": "A", "available": true } }
  },
  "module_results": { ... },
  "all_issues": [ ... ],
  "recommendations": [ ... ],
  "flight_id": 42,
  "html_report_url": "/reports/flight_42_report.html"
}
```

## 11.4 GET /api/profiles/{vehicle\_type}

Returns all editable profile fields for the UI settings panel.

```
curl http://localhost:5000/api/profiles/quadplane
```

## 11.5 GET /api/fleet/{drone\_id}/history

Returns: - `drone_id`, `drone_type`, `total_flights` - `history[]` — array of flight records - `trends{}` — trend objects keyed by metric name - `alerts[]` — active alert message strings

---

## 12. Troubleshooting & FAQ

### 12.1 Common Errors

| Error                                      | Cause                                  | Resolution                                                                        |
|--------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------|
| Only <code>.BIN</code> files are supported | Wrong file type uploaded               | Upload raw DataFlash <code>.BIN</code> from SD card                               |
| Drone ID is required                       | Empty drone ID field                   | Enter a non-empty drone name                                                      |
| Unknown vehicle type                       | Invalid vehicle_type value             | Use <code>quadcopter</code> , <code>quadplane</code> , or <code>fixed_wing</code> |
| Port 5000 already in use                   | Previous server instance still running | Change port in <code>run_web.py</code> :<br><code>port=5001</code>                |
| No module named <code>'pymavlink'</code>   | Dependencies not installed             | Run <code>pip install -r requirements.txt</code>                                  |
| <code>PARSE ERROR</code> in CLI            | Corrupt or non-DataFlash file          | Re-export raw <code>.BIN</code> from flight controller SD card                    |
| Module shows <code>available: false</code> | Required log messages missing          | See Section 12.2 below                                                            |

### 12.2 Module Unavailability

Some modules require specific ArduPilot logging parameters to be enabled:

| Module                  | Cause of Unavailability | How to Enable                                          |
|-------------------------|-------------------------|--------------------------------------------------------|
| <code>pid_tuning</code> | PIDR/PIDP not in log    | Set <code>LOG_BITMASK</code> bit 6 in ArduPilot params |
| <code>telemetry</code>  | RADIO messages absent   | Connect SiK/RFD900 radio; enable RADIO logging         |



|                              |                             |                                                     |
|------------------------------|-----------------------------|-----------------------------------------------------|
| <code>esc_telemetry</code>   | ESC messages absent         | Configure BLHeli32/AM32 ESC telemetry passthrough   |
| <code>airspeed</code>        | Multirotor type             | Module not applicable; returns score 100            |
| <code>vtol_transition</code> | Non-VTOL type               | Module not applicable; returns score 100            |
| <code>mission</code>         | No CMD data or no AUTO mode | Ensure CMD messages are logged; fly an AUTO mission |
| <code>rc_link</code>         | No RCIN/RSSI messages       | Check RC logging configuration                      |

## 12.3 Port Conflict Resolution

If port 5000 is taken:

```
# In run_web.py, change:
uvicorn.run(app, host="0.0.0.0", port=5000)
# To:
uvicorn.run(app, host="0.0.0.0", port=5001)
```

Then open `http://localhost:5001`.

## 12.4 Frequently Asked Questions

### Q: Which log format does the tool support?

A: ArduPilot DataFlash `.BIN` logs only. These come from the SD card on the flight controller. `.tlog` (telemetry logs) and `.log` (text logs) are not supported.

### Q: Does the tool send data to any cloud service?

A: No. All processing and storage is local. No internet connection is used during analysis.

### Q: Why does the overall score change slightly after hardware replacement?

A: The score reflects actual flight data against your profile thresholds. After replacement, update the profile if component specs changed (e.g. new battery capacity or motor type).

### Q: How many flights are needed before trend graphs appear?

A: Minimum 3 flights per drone. Trend regression requires at least 3 data points.

### Q: Can I customize the scoring weights?

A: Yes. Edit the `scoring_weights` section in your YAML profile file. Weights must sum to 1.0.

**Q: Why does my VTOL show available=false for vtol\_transition?**

A: Ensure you selected **VTOL Hybrid** (not Quadcopter) and uploaded a QuadPlane log. Check that the log contains Q-mode entries and fixed-wing cruise segments.

**Q: Can I run multiple analyses at the same time?**

A: The web server processes one analysis request at a time per upload. Do not submit multiple concurrent analysis requests.

**Q: How do I update the tool to the latest version?**

```
git pull
pip install -r requirements.txt
python run_web.py
```

---

## 13. Glossary

| Term                           | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| ArduPilot                      | Open-source autopilot firmware for multirotors, fixed-wing, VTOL, and more                                                           |
| DataFlash <a href="#">.BIN</a> | Binary onboard flight log produced by ArduPilot on the SD card                                                                       |
| Module                         | Independent analysis component (e.g. battery, motors, vibration)                                                                     |
| Overall Score                  | Weighted aggregate score from 0 to 100 derived from all module scores                                                                |
| Grade                          | Letter grade A–F derived from overall score                                                                                          |
| Issue Code                     | Stable identifier for a detected condition (e.g. <a href="#">BAT-002</a> )                                                           |
| Severity                       | Issue criticality: <a href="#">critical</a> (ground aircraft), <a href="#">warning</a> (investigate), <a href="#">info</a> (context) |
| Profile                        | YAML file defining vehicle-specific thresholds, expectations, and scoring weights                                                    |
| Override                       | Runtime profile value substitution sent from UI or API without editing YAML                                                          |
| Fleet History                  | Per-drone list of all analyzed flights stored in the local SQLite database                                                           |
| Trend Slope                    | Per-flight change rate for a metric, computed via linear regression                                                                  |
| Availability Flag              | Module field ( <a href="#">available</a> ) indicating whether required log data existed                                              |

|             |                                                                             |
|-------------|-----------------------------------------------------------------------------|
| ARSP        | Dedicated airspeed sensor log message in ArduPilot                          |
| VTOL        | Vertical Take-Off and Landing aircraft (e.g. QuadPlane)                     |
| EKF         | Extended Kalman Filter — ArduPilot sensor fusion algorithm                  |
| HDOP        | Horizontal Dilution of Precision — GPS accuracy indicator (lower is better) |
| BEC         | Battery Eliminator Circuit — voltage regulator for FC/servo power rail      |
| ESC         | Electronic Speed Controller — drives each motor from PWM or CAN signal      |
| LOG_BITMASK | ArduPilot parameter controlling which message types are written to the log  |
| RCOU        | Radio Control Output — logged PWM values sent to each motor/servo channel   |

---

*UAV Flight Analyzer — Internal Technical Documentation*

*All analysis is performed locally. No data is transmitted externally.*